

kci-dev:

What Changed, What Works, and What Kernel Developers Still Need

Arisu Tachibana

kci-dev creator and project lead

Kernel Testing & Dependability MC / LPC 2026

Our work since LPC 2025

- **Reliable automation:** fixed hangs in job watching and bisection.
- **Clearer results:** corrected false success reports, validation and JSON output.
- **Python API:** lets existing tools use kci-dev without parsing terminal output.
- **Broader workflows:** added external build reporting and patch submission.

A maintainer's question

What changed for my kernel?

- Which revision and configuration did we test?
- Which failures appeared with my change?
- Do we have enough evidence to act?

kci-dev brings KernelCI data into scripts and review workflows.

What changed since LPC 2025

Available by v0.1.11	What it enables
Python client	Import kci-dev into other tools
KCIDB submission + storage	Report external builds and upload artifacts
Patchset submission	Test local patches or allowed patch URLs
Validation + workflow fixes	Check consistency and improve automation

Current main: structured comparison reports and CI gates

Snapshot: 4 October 2026, `e4c0087`. MCP remains experimental.

A reusable Python interface

```
from kcidb import KernelCIClient

client = KernelCIClient()
summary = client.get_summary(
    origin="maestro", giturl=GIT_URL,
    branch=BRANCH, commit=COMMIT,
)
```

Dashboard queries return Python objects.

`submit_build(...)` reports an external build to KCIDB.

Patches on a known base

```
kci-dev patchset --nodeid "$CHECKOUT_NODE" \  
  --patch 0001-fix.patch \  
  --patch 0002-test.patch \  
  --job-filter "$JOB_FILTER" \  
  --watch --test "$TEST_PATH"
```

Requires an existing checkout and a configured pipeline/token.

`--patchurl` accepts allowed URLs, including Patchwork mbox URLs.

Comparing two revisions

```
kci-dev results compare \  
  --giturl "$GIT_URL" --branch "$BRANCH" \  
  --format json "$BASE" "$HEAD"
```

Current main: explicit full commit hashes and structured output

Matches origin, platform, arch, compiler, config and path.

Keeps repeated results and their IDs.

What the report tells us

Category	Interpretation
<code>regression</code>	PASS becomes FAIL/ERROR, or history signal
<code>fixed</code>	Recovery to PASS
<code>persistent_fail</code>	Failure on both revisions
<code>unstable</code>	History signal or another status change
<code>new</code>	Result appears only in HEAD
<code>missing</code>	Result appears only in BASE

A regression label needs investigation.

`FAIL` and `ERROR` both count as failures in this classifier.

Gate policy and coverage

```
kci-dev results gate \  
  --giturl "$GIT_URL" --branch "$BRANCH" \  
  --base "$BASE" --head "$HEAD" \  
  --fail-on regression,missing --format json
```

Exit	Meaning in current main
0	No selected category triggered the policy
1	A selected category triggered the policy
2	Incomplete comparison or command error

Expected coverage still needs an explicit test plan.

Reliability across services

- `maestro validate` checks build and boot consistency
- Current comparison still needs stronger history scoping
- CLI instance selection needs consistent propagation

One review should retain its revision, endpoint and evidence.

MCP as an experimental interface

```
pip install 'kci-dev[mcp]'  
kci-dev mcp
```

- Query results, hardware and known issues
- Inspect Maestro nodes with configured API access
- Trigger or retry jobs with pipeline access and a token

Tool names and response formats can change.

Proposed priorities

Workstream	Concrete next contribution
Comparison correctness	Pin history, propagate config, test incomplete data
Patch review	Track series version, base and result IDs
Reproducible test plans	Record expected coverage and completion rules

Follow-up: richer lab metadata, caching and multi-tree reporting

Decisions for this MC

- Which workflow should we finish first?
Patch review or release review?
- What evidence would you require to trust it?
- Who can bring a real tree or series and help validate it?

kci.dev

github.com/kernelci/kci-dev